



DIVISION OF PHYSICAL SCIENCES AND MATHEMATICS

College of Arts and Sciences | University of the Philippines Visayas

Miagao, Iloilo, 5023, Philippines | Tel. No. (033) 513-7020

Email Address: psm.upvisayas@up.edu.ph

COURSE MATERIAL

Coin Dash Pt. 2

CMSC 197 GDD Activity 3

Prepared by: Rene Andre Bedonia Jocsing

Published: February 03, 2026

Session Objectives

- Build reusable collectible scenes using scene inheritance patterns
- Implement tweens for polished pickup animations
- Create spawning systems with randomized positioning
- Integrate game timer and loss condition logic
- Complete the core game loop architecture

Part 1: Coin Scene

The coin is a simple collectible that disappears when touched. However, we'll add visual polish through tweens to make collection satisfying.

Scene Setup

Node Structure:

```
1 Coin (Area2D)
2 |-- AnimatedSprite2D
3 |-- CollisionShape2D
4 `-- Timer
```

Build Steps:

1. Create new scene, root: Area2D named Coin
2. Save as coin.tscn
3. Add AnimatedSprite2D child
4. Add CollisionShape2D child
5. Add Timer child, rename to Lifetime

Configure Collision

- Coin Layer: collectibles (layer 4)
- Coin Mask: player (layer 2)
- Node Tab, Groups: Add coins

Design Decision: Collision Sizing

Make collectible hitboxes slightly **larger** than the sprite. This reduces player frustration and makes collection feel more forgiving. The opposite is true for enemies: make their hitboxes slightly smaller.

Animation Setup

In AnimatedSprite2D:

1. Frames: New SpriteFrames
2. Create animation (any name, default is fine)
3. Navigate to res://assets/coin
4. Drag frames into the animation
5. Set Speed: 6 FPS, Looping: Off (we will randomize our timings)
6. Scale sprite: (0.4, 0.4)

For CollisionShape2D:

- Shape: New CircleShape2D
- Size slightly larger than sprite

Coin Script

```
1 extends Area2D
2
3 var screensize: Vector2 = Vector2.ZERO
4
5 func pickup() -> void:
6     # Disable collision immediately to prevent double-pickup
7     $CollisionShape2D.set_deferred("disabled", true)
8
9     # Create tween for scale and fade effects
10    var tween := create_tween().set_parallel().set_trans(
11        Tween.TRANS_QUAD)
12    tween.tween_property(self, "scale", scale * 3, 0.3)
13    tween.tween_property(self, "modulate:a", 0.0, 0.3)
14
15    # Wait for animation, then remove
16    await tween.finished
17    queue_free()
```

Design Pattern: Tweens for Game Feel

Tweens interpolate values over time using easing functions. Here we use:

- `set_parallel()`: Both animations play simultaneously
- `set_trans(Tween.TRANS_QUAD)`: Quadratic easing curve
- `tween_property()`: Animates object properties
- `await tween.finished`: Waits before cleanup

Why disable collision first? Prevents player from triggering pickup twice during the tween.

Common Pitfalls

Common mistakes:

- Forgetting `set_deferred()` on collision shape (causes physics errors)
- Calling `queue_free()` immediately (tween won't play)
- Not using `await` (node deleted before animation completes)

Part 2: Powerup Scene

Powerups use identical architecture to coins but with different visuals and effects. We'll leverage scene duplication.

Create via Duplication

1. Open `coin.tscn`
2. Scene, Save Scene As: `powerup.tscn`
3. Rename root node to `Powerup`
4. Node Tab, Groups: Remove `coins`, add `powerups`

Update Animation

- `AnimatedSprite2D`, Frames: Use existing `SpriteFrames`
- Delete coin frames
- Add frames from `res://assets/sprites/pow`
- Verify frame Speed: 6 FPS

Modify Script

Script remains identical to `coin` except for potential effects. For now, keep same tween behavior. The player script will handle different outcomes via groups.

```

1  extends Area2D
2
3  var screensize: Vector2 = Vector2.ZERO
4
5  func pickup() -> void:
6      $CollisionShape2D.set_deferred("disabled", true)
7
8      var tween := create_tween().set_parallel().set_trans(
9          Tween.TRANS_QUAD)
10     tween.tween_property(self, "scale", scale * 3, 0.3)
11     tween.tween_property(self, "modulate:a", 0.0, 0.3)
12
13     await tween.finished
14     queue_free()

```

Can we somehow use some OOP to produce cleaner code?

Part 3: Main Scene Architecture

Main scene orchestrates spawning, game state, and win/loss conditions.

Scene Setup**Node Structure:**

```

1  Main (Node)
2  |-- Player (instance)
3  |-- Background (TextureRect)
4  `-- GameTimer (Timer)

```

Build Steps:

1. New scene, root: Node named `Main`
2. Add instance of `player.tscn`
3. Add `TextureRect` named `Background`
 - Layout: Full Rect
 - Texture: `res://assets/grass.png`
 - Stretch Mode: Tile

- Drag to top of node tree (drawn first)
- Resize so it fits our whole screen

4. Add Timer named GameTimer

Main Script: Core Variables

```

1  extends Node
2
3  @export var coin_scene: PackedScene
4  @export var powerup_scene: PackedScene
5  @export var playtime: int = 30
6
7  var level: int = 1
8  var score: int = 0
9  var time_left: int = 0
10 var screensize: Vector2 = Vector2.ZERO
11 var playing: bool = false

```

Link Scenes in Inspector:

- Drag coin.tscn to Coin Scene property
- Drag powerup.tscn to Powerup Scene property

Initialization

```

1  func _ready() -> void:
2      screensize = get_viewport().get_visible_rect().size
3      $Player.screensize = screensize
4      $Player.hide()

```

Architecture Note: Hiding Player

Player is hidden during menu/game-over states. The `new_game()` function will show and position it. This prevents the player from appearing before the game starts.

Spawning System

```

1  func spawn_coins() -> void:
2      for i in level + 4:
3          var coin := coin_scene.instantiate()
4          add_child(coin)
5          coin.screensize = screensize
6          coin.position = Vector2(
7              randi_range(0, screensize.x),
8              randi_range(0, screensize.y)
9          )

```

Design Pattern: Dynamic Difficulty Scaling

Formula: `level + 4`

- Level 1: 5 coins
- Level 2: 6 coins
- Level 3: 7 coins

This creates progressive difficulty without overwhelming the player. Adjust the constant (4) to tune difficulty curve.

Game Loop Logic

```

1  func new_game() -> void:
2      playing = true
3      level = 1
4      score = 0
5      time_left = playtime
6
7      $Player.start()
8      $Player.show()
9      $GameTimer.start()
10     spawn_coins()
11
12
13  func _process(delta: float) -> void:
14      if not playing:
15          return
16
17      # Check if all coins collected
18      if get_tree().get_nodes_in_group("coins").size() == 0:
19          level += 1
20          time_left += 5
21          spawn_coins()

```

Design Pattern: Scene Tree Queries

`get_tree().get_nodes_in_group("coins")` returns all nodes in the “coins” group. This is more flexible than tracking count manually:

- Automatically accounts for destroyed coins
- Works with dynamically spawned objects
- No need to maintain separate counter variable

Timer Integration

Configure GameTimer:

- Wait Time: 1.0
- One Shot: Off (repeats every second)
- Autostart: Off (manual control)

Connect timeout signal:

```

1  func _on_game_timer_timeout() -> void:
2      time_left -= 1
3      if time_left <= 0:
4          game_over()
5
6
7  func game_over() -> void:
8      playing = false
9      $GameTimer.stop()
10     get_tree().call_group("coins", "queue_free")
11     $Player.die()

```

Design Pattern: Group Cleanup

`call_group("coins", "queue_free")` calls `queue_free()` on every node in the group. This is cleaner than:

- Iterating manually through children
- Maintaining references to spawned objects
- Checking node types individually

Part 4: Player Integration

Update player script to handle powerups differently from coins.

Modify Player Signals

```

1  # Change signal emission to include type
2  func _on_area_entered(area: Area2D) -> void:
3      if area.is_in_group("coins"):
4          area.pickup()
5          pickup.emit("coin")
6      if area.is_in_group("powerups"):
7          area.pickup()
8          pickup.emit("powerup")
9      if area.is_in_group("obstacles"):
10         hurt.emit()
11         die()

```

What design pattern is this? Can we do better?

Update Main to Handle Types

Connect Player signals:

1. Select Player instance in Main scene
2. Node tab, find pickup signal
3. Connect to Main script

```

1  func _on_player_pickup(type: String) -> void:
2      match type:
3          "coin":
4              score += 1
5          "powerup":
6              time_left += 5

```

Design Pattern: Match for Type Handling

`match` statements are cleaner than `if-elif` chains when checking a single variable against multiple values. Benefits:

- More readable for multiple cases
- Easier to extend with new types
- Underlying C++ compiler can optimize better

Testing Checklist

Core Functionality

- Player can collect coins (score increases)

- Coins play tween animation on pickup
- Timer counts down every second
- Game ends when timer reaches 0
- All remaining coins removed on game over

Level Progression

- Collecting all coins advances level
- New coins spawn with increased count
- Timer bonus awarded on level up

Common Pitfalls

Common issues:

- Timer not starting: Check `new_game()` calls `$GameTimer.start()`
- Coins not spawning: Verify scene is linked in Inspector
- Double pickups: Ensure collision disabled in `pickup()`
- Level not advancing: Check group name is “coins” exactly

Architecture Review

What We Built

- Reusable collectible architecture (coin/powerup scenes)
- Dynamic spawning system with scaling difficulty
- Signal-driven game state transitions
- Timer-based loss condition
- Group-based object management

Key Takeaways

1. **Scene Reusability:** Powerup duplicated from Coin saves time
2. **Tweens for Polish:** Small animations dramatically improve feel
3. **Group Queries:** Flexible way to track and manipulate objects
4. **Match Statements:** Clean type handling for similar objects

Next Session Preview

- Build HUD with signal-driven updates
- Implement title screen and menu state
- Add sound effects, background music, and visual polish
- Complete full game loop with restart capability